



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi and Affiliated to JNTUH)

Narayanaguda, Hyderabad – 500029

Department Of Information Technology

INFORMATION SECURITY LAB MANUAL

SUBJECT :23IT702PC

REGULATION: KR23

CLASS: IV B.TECH I SEM

(2026-2027)



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)**



Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Department Of Information Technology

Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

_____ bearing Roll No _____ of _____

Branch of _____ year B.Tech Course in the _____

Subject during the Academic year _____ & _____ under our supervision.

Number of week tasks completed: _____

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner



S.No.	Name of	Date	Observation	Record	Viva	Total	FacultySignature
	TOTAL						



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and Affiliated to JNTUH)
3-5-1026, Narayanaguda, Hyderabad-29. Ph: 040-23261407



Department Of Information Technology

Vision of the Institution:

- To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and Affiliated to JNTUH)
3-5-1026, Narayanaguda, Hyderabad-29. Ph: 040-23261407



Department Of Information Technology

Vision & Mission of Department

Vision of the Department:

- To produce globally competent graduates to meet the modern challenges through contemporary knowledge and moral values committed to build a vibrant nation.

Mission of the Department:

- To create an academic environment, this promotes the intellectual and professional development of students and faculty.
- To impart skills beyond university prescribed to transform students into a well-rounded IT professional.
- To nurture the students to be dynamic, industry ready and to have multidisciplinary skills including e-learning, blended learning and remote testing as an individual and as a team.
- To continuously engage in research and projects development, strategic use of emerging technologies to attain self-sustainability.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and Affiliated to JNTUH)
3-5-1026, Narayanaguda, Hyderabad-29. Ph: 040-23261407



Department of Information Technology

PROGRAM OUTCOMES (POs)

- **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- **The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

- **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and
Affiliated to JNTUH) 3-5-1026, Narayanaguda,
Hyderabad-29. Ph: 040-23261407

Department of Information Technology

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(Approved by AICTE & Govt of T.S and Affiliated to JNTUH)
3-5-1026, Narayanaguda, Hyderabad-29. Ph: 040-23261407

Department of Information Technology

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and
Affiliated to JNTUH) 3-5-1026, Narayanaguda,
Hyderabad-29. Ph: 040-23261407

Department Of Information Technology

SYLLABUS

IT703PC: INFORMATION SECURITY LAB

IV Year B.Tech. IT I –SEM

L T P C

0 0 2 1

List of Experiments:

1. Write a C program that contains a string (char pointer) with a value 'Hello world'. The program should XOR each character in this string with 0 and displays the result.
2. Write a C program that contains a string (char pointer) with a value 'Hello world'. The program should AND or and XOR each character in this string with 127 and display the result.
3. Write a Java program to perform encryption and decryption using the following algorithms
a. Ceaser cipher b. Substitution cipher c. Hill Cipher
4. Write a C/JAVA program to implement the DES algorithm logic.
5. Write a C/JAVA program to implement the Blowfish algorithm logic.
6. Write a C/JAVA program to implement the Rijndael algorithm logic.
7. Write the RC4 logic in Java Using Java cryptography; encrypt the text "Hello world" using Blowfish. Create your own key using Java key tool.
8. Write a Java program to implement RSA algorithm.
9. Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript.
10. Calculate the message digest of a text using the SHA-1 algorithm in JAVA.
11. Calculate the message digest of a text using the MD5 algorithm in JAVA.

[illegible]



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S and
Affiliated to JNTUH) 3-5-1026, Narayanaguda,
Hyderabad-29. Ph: 040-23261407

Department Of Information Technology

TABLE OF CONTENTS

S.NO	NAME OF THE EXPERIMENT	PAGE NUMBERS
1	Write a C program that contains a string (char pointer) with a value 'Hello world'. The program should XOR each character in this string with 0 and displays the result.	1
2	Write a C program that contains a string (char pointer) with a value 'Hello world'. The program should AND or and XOR each character in this string with 127 and display the result.	2-3
3	Write a Java program to perform encryption and decryption using the following algorithms a. Ceaser cipher b. Substitution cipher c. Hill Cipher	4-12
4	Write a C/JAVA program to implement the DES algorithm logic.	13-16
5	Write a C/JAVA program to implement the Blowfish algorithm logic.	17-18
6	Write a C/JAVA program to implement the Rijndael algorithm logic.	19-20
7	Write the RC4 logic in Java Using Java cryptography; encrypt the text "Hello world" using Blowfish. Create your own key using Java key tool.	21-22
8	Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript	25-27
9	Calculate the message digest of a text using the SHA-1 algorithm in JAVA.	28-30
10	Calculate the message digest of a text using the MD5 algorithm in JAVA.	31-33

EXPERIMENT - 1

AIM: Write a C program that contains a string (char pointer) with a value 'Hello world'.
The program should XOR each character in this string with 0 and displays the result.

DESCRIPTION:

An XOR gate is a digital logic gate with two or more inputs and one output that performs exclusive disjunction.

ALGORITHM:

1. Start
2. Take input as "Hello World" which is assigned to variable "str"
3. Initialise the variable 'len' for calculating the length of the string
4. Print the length of the word followed by the input

PROGRAM:

```
#include<stdlib.h>

int main(){
    char str[]="Hello World";
    char str1[11];
    int i,len;
    len=strlen(str);
    for(i=0;i<len;i++) {
        str1[i]=str[i]^0;
        printf("%c",str1[i]);
    }
    printf("\n");
}
```

OUTPUT:

Hello World

Hello World

EXPERIMENT - 2

AIM: Write a C program that contains a string (char pointer) with a value 'Hello world'. The program should AND or and XOR each character in this string with 127 and display the result.

DESCRIPTION:

The AND gate is an electronic circuit that gives a high output(1) only if all the inputs are high. A dot(.) is used to show the AND operation i.e. A.B this dot is sometimes omitted.

ALGORITHM:

1. Start
2. Take the input 'hello world' which is assigned to variable 'str'
3. Perform AND operation between the string and 127.
4. Then print the result
5. Stop.

PROGRAM:

```
#include <stdio.h>
#include<stdlib.h>
void main(){
    char str[]="Hello World";
    char str1[11];
    char str2[11]=str[];
    int i,len;
    len = strlen(str);
    for(i=0;i<len;i++) {
        str1[i] = str[i]&127;
        printf("%c",str1[i]);
    }
    printf("\n");
```

```
        for(i=0;i<len;i++) {  
            str3[i] = str2[i]^127;  
            printf("%c",str3[i]);  
        }  
        printf("\n");  
    }  
}
```

Output:

Hello World

Hello World

Hello World

EXPERIMENT – 3

AIM: Write a Java program to perform encryption and decryption using the following algorithms

a. Ceaser cipher b. Substitution cipher c. Hill Cipher

a) Ceaser Cipher

DESCRIPTION:

The Caesar cipher technique is one of the earliest and simplest method of encryption technique .It's simple type of substitution cipher i.e. each letter of given text is replaced by a letter, some fixed number of positions down the alphabet. Thus, to cipher a given text, we need an integer value, known as shift which indicates the number of position each letter of the text has been moved down. The encryption can be replaced by using modular arithmetic by first transforming the letter into numbers according to the scheme A=0,B=1,... Y=24,Z=25.

e.g. ABCD; shift=4 Cipher: EFGH

ALGORITHM:

1. Start
2. Read the string
3. Traverse the given text one character at a time
4. For each character transforms the given character as per the key
5. It prints the encryption and decryption of the given string
6. Stop

PROGRAM:

```
import java.util.Scanner;
```

```
public class CaesarCipher {
```

```
    // Method for Encryption
```

```
    public static String
```



```

encrypt(String text, int key) {

    StringBuilder result = new
    StringBuilder();

    for (int i = 0; i <
    text.length(); i++) {

        char ch = text.charAt(i);

        if
        (Character.isUpperCase(ch)) {

            ch = (char) (((ch - 'A' +
            key) % 26) + 'A');

            } else if
            (Character.isLowerCase(ch)) {

                ch = (char) (((ch - 'a' +
                key) % 26) + 'a');

                }

            result.append(ch);

        }

        return result.toString();

    }

    // Method for Decryption

    public static String
    decrypt(String text, int key) {

```

```

        StringBuilder result = new
StringBuilder();

        for (int i = 0; i <
text.length(); i++) {

            char ch = text.charAt(i);

            if
(Character.isUpperCase(ch)) {

                ch = (char) (((ch - 'A' -
key + 26) % 26) + 'A');

            } else if
(Character.isLowerCase(ch)) {

                ch = (char) (((ch - 'a' -
key + 26) % 26) + 'a');

            }

            result.append(ch);

        }

        return result.toString();

    }

    public static void main(String[]
args) {

        Scanner sc = new
Scanner(System.in);

        System.out.print("Enter Plain
Text: ");

```

```

        String plainText =
sc.nextLine();

        System.out.print("Enter Key
(Shift Value): ");

        int key = sc.nextInt();

        String cipherText =
encrypt(plainText, key);

System.out.println("Encrypted
Text: " + cipherText);

        String originalText =
decrypt(cipherText, key);

System.out.println("Decrypted
Text: " + originalText);

        sc.close();

    }
} import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Scanner;
public class CeaserCipher {

```

```

static Scanner sc=new Scanner(System.in);
static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
public static void main(String[] args) throws IOException {
    // TODO code application logic here
        System.out.print("Enter any String: ");
        String str = br.readLine();
        System.out.print("\nEnter the Key: ");
        int key = sc.nextInt();
        String encrypted = encrypt(str, key);
        System.out.println("\nEncrypted String is: " +encrypted);
        String decrypted = decrypt(encrypted, key);
        System.out.println("\nDecrypted String is: " +decrypted);
        System.out.println("\n\n");
    }
    public static String encrypt(String str, int key) {
        String encrypted = "";
        for(int i = 0; i < str.length(); i++) {
            int c = str.charAt(i);
            if (Character.isUpperCase(c)) {
                c = c + (key % 26);
                if (c > 'Z')
                    c = c - 26;
            }
            else if (Character.isLowerCase(c)) {
                c = c + (key % 26);
                if (c > 'z')
                    c = c - 26;
            }
            encrypted += (char) c;
        }
        return encrypted;
    }
}

```

```

    }
    public static String decrypt(String str, int key) {
        String decrypted = "";
        for(int i = 0; i < str.length(); i++) {
            int c = str.charAt(i);
            if(Character.isUpperCase(c)) {
                c = c - (key % 26);
                if (c < 'A')
                    c = c + 26;
            }
            else if(Character.isLowerCase(c)) {
                c = c - (key % 26);
                if (c < 'a')
                    c = c + 26;
            }
            decrypted += (char) c;
        }
        return decrypted;
    }
}

```

OUTPUT:

Enter Plain Text: MEMORIAL

Enter Key (Shift Value): 3

Encrypted Text: PHPRULDO

Decrypted Text: MEMORIAL

b) Substitution Cipher

DESCRIPTION:

Substitution Cipher is a method of encrypting by which units of plain text are replaced with cipher text, the 'units' may be single letter or mixture of the letters. In a substitution cipher, the units of the plain text are retained in the same sequence in the cipher text, but the units themselves are altered.

ALGORITHM:

1. Start
2. Assign a= 'abcdefghijklmnopqrstuvwxyz' b= 'zyxwvutsrqponmkjihg fedcba'
3. Read input string which need to be encrypted
4. Compare variables 'a' and 'b' values and substitute the input string position with variable 'b' strings till end of input string
5. Print cipher text which is produced from step-4
6. Stop

PROGRAM:

```
import java.io.*;
import java.util.*;

public class SubstitutionCipher {

    static Scanner sc = new Scanner(System.in);
    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String[] args) throws IOException {

        String a = "abcdefghijklmnopqrstuvwxyz";
        String b = "zyxwvutsrqponmlkjihgfedcba";

        System.out.print("Enter any string: ");
        String str = br.readLine();

        String encrypt = "";
        char c;

        for (int i = 0; i < str.length(); i++) {
            c = str.charAt(i);

            int j = a.indexOf(c);
```

```
if (j != -1)
    encrypt = encrypt + b.charAt(j);
else
    encrypt = encrypt + c; // Keeps spaces and symbols unchanged
}

System.out.println("The encrypted data is: " + encrypt);
}
}
```

Output

Enter any string: hello
The encrypted data is: svool

c) Hill Cipher

DESCRIPTION:

Hill cipher is a polygraph substitution cipher based on linear algebra. Each letter is represented by a number modulo 26. To encrypt a message, each block of 'n' letters is multiplied by an invertible $n \times n$ matrix against modulus 26. To decrypt the message, each block is multiplied by the inverse of the matrix used for encryption. The matrix used for encryption is the cipher key and it should be chosen randomly from the set of invertible $n \times n$.

ALGORITHM:

1. Start
2. To encrypt a string each block of '3' letters are multiplied by invertible 3×3 matrix against %26
3. To decrypt a string each block is multiplied by inverse of the matrix used for encryption
4. Read a three-letter string
5. It prints the inverse matrix, encryption and decryption of the given three letter string
6. Stop.

PROGRAM:

```
import java.util.Scanner;

public class HillCipher {

    static final int SIZE = 3;
    static final int MOD = 26;

    // Find modular inverse of a number
    static int modInverse(int a) {
        a = ((a % MOD) + MOD) % MOD;
        for (int i = 1; i < MOD; i++) {
            if ((a * i) % MOD == 1)
                return i;
        }
        return -1;
    }

    // Find determinant of 3x3 matrix
    static int determinant(int[][] key) {
        int det = key[0][0] * (key[1][1] * key[2][2] - key[1][2] * key[2][1])
            - key[0][1] * (key[1][0] * key[2][2] - key[1][2] * key[2][0])
            + key[0][2] * (key[1][0] * key[2][1] - key[1][1] * key[2][0]);
    }
}
```



```

det = ((det % MOD) + MOD) % MOD;
    return det;
}

// Find inverse of key matrix modulo 26
static int[][] inverseMatrix(int[][] key) {

    int det = determinant(key);
    int detInv = modInverse(det);

    if (detInv == -1) {
        System.out.println("Key matrix is not invertible.");
        System.exit(0);
    }

    int[][] inv = new int[SIZE][SIZE];

    inv[0][0] = key[1][1] * key[2][2] - key[1][2] * key[2][1];
    inv[0][1] = -(key[0][1] * key[2][2] - key[0][2] * key[2][1]);
    inv[0][2] = key[0][1] * key[1][2] - key[0][2] * key[1][1];

    inv[1][0] = -(key[1][0] * key[2][2] - key[1][2] * key[2][0]);
    inv[1][1] = key[0][0] * key[2][2] - key[0][2] * key[2][0];
    inv[1][2] = -(key[0][0] * key[1][2] - key[0][2] * key[1][0]);

    inv[2][0] = key[1][0] * key[2][1] - key[1][1] * key[2][0];
    inv[2][1] = -(key[0][0] * key[2][1] - key[0][1] * key[2][0]);
    inv[2][2] = key[0][0] * key[1][1] - key[0][1] * key[1][0];

    // Transpose (Adjoint) and multiply by determinant inverse
    int[][] result = new int[SIZE][SIZE];

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            result[i][j] = ((inv[j][i] * detInv) % MOD + MOD) % MOD;
        }
    }

    return result;
}

// Matrix multiplication modulo 26
static int[] multiply(int[][] matrix, int[] text) {

    int[] result = new int[SIZE];

    for (int i = 0; i < SIZE; i++) {
        result[i] = 0;
        for (int j = 0; j < SIZE; j++) {
            result[i] += matrix[i][j] * text[j];
        }
    }
}

```

```

        result[i] = ((result[i] % MOD) + MOD) % MOD;
    }

    return result;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int[][] key = new int[SIZE][SIZE];

    System.out.println("Enter 3x3 Key Matrix:");

    for (int i = 0; i < SIZE; i++)
        for (int j = 0; j < SIZE; j++)
            key[i][j] = sc.nextInt();

    System.out.print("Enter 3-letter plaintext: ");
    String text = sc.next().toLowerCase();

    if (text.length() != 3) {
        System.out.println("Plaintext must contain exactly 3 letters.");
        return;
    }

    int[] plain = new int[SIZE];

    for (int i = 0; i < SIZE; i++)
        plain[i] = text.charAt(i) - 'a';

    // Encryption
    int[] cipher = multiply(key, plain);

    System.out.print("Encrypted Text: ");

    for (int i = 0; i < SIZE; i++)
        System.out.print((char) (cipher[i] + 'a'));

    System.out.println();

    // Decryption
    int[][] inverse = inverseMatrix(key);

    int[] decrypted = multiply(inverse, cipher);

    System.out.print("Decrypted Text: ");

    for (int i = 0; i < SIZE; i++)
        System.out.print((char) (decrypted[i] + 'a'));
}

```

```
        System.out.println();  
        sc.close();  
    }  
}
```

Sample Input

Enter 3x3 Key Matrix:

6 24 1

13 16 10

20 17 15

Enter 3-letter plaintext:

act

Sample Output

Encrypted Text: poh

Decrypted Text: act

EXPERIMENT – 4

AIM: Write a C/JAVA program to implement the DES algorithm logic.

DESCRIPTION:

DES is an implementation of Feistel Cipher. It uses 16 round Feistel structure. The block size is 64 bits. Though, key length is 64-bits, DES has an effective key length and 56-bits. Since 8 of the 64bits of the key one not used by the encryption algorithm.

ALGORITHM:

1. First we need to get the key generator instance using DES algorithm.
2. Generate secure key that will be used for encryption and decryption.
3. Get Cipher instance using DES algorithm. One for encrypt mode and another for decrypt mode. Initialize the cipher object using key and IVParameterSpec object.
4. For encryption, create object of CipherOutputStream using encrypt cipher. For decryption, create object of CipherInputStream using decrypt cipher.
5. Read the input stream and write the output stream.
6. Stop

PROGRAM:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Base64;

import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESedeKeySpec;

public class DES {

    private static final String UNICODE_FORMAT = "UTF8";
    public static final String DESEDE_ENCRYPTION_SCHEME = "DESEde";

    private DESedeKeySpec myKeySpec;
    private SecretKeyFactory mySecretKeyFactory;
    private Cipher cipher;
    private SecretKey key;

    private String myEncryptionKey;
    private String myEncryptionScheme;
```

```

static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

public DES() throws Exception {

    myEncryptionKey = "ThisIsSecretEncryptionKey";
    myEncryptionScheme = DESEDE_ENCRYPTION_SCHEME;

    byte[] keyAsBytes = myEncryptionKey.getBytes(UNICODE_FORMAT);

    myKeySpec = new DESedeKeySpec(keyAsBytes);
    mySecretKeyFactory = SecretKeyFactory.getInstance(myEncryptionScheme);
    cipher = Cipher.getInstance(myEncryptionScheme);

    key = mySecretKeyFactory.generateSecret(myKeySpec);
}

public String encrypt(String plainText) {

    String encryptedString = "";

    try {
        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] plainTextBytes = plainText.getBytes(UNICODE_FORMAT);
        byte[] encryptedBytes = cipher.doFinal(plainTextBytes);

        encryptedString = Base64.getEncoder().encodeToString(encryptedBytes);

    } catch (Exception e) {
        e.printStackTrace();
    }

    return encryptedString;
}

public String decrypt(String encryptedText) {

    String decryptedText = "";

    try {

        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] encryptedBytes = Base64.getDecoder().decode(encryptedText);

        byte[] plainText = cipher.doFinal(encryptedBytes);

        decryptedText = new String(plainText, UNICODE_FORMAT);

    }
}

```

```

catch (Exception e) {
    e.printStackTrace();
}

return decryptedText;
}

public static void main(String[] args) throws Exception {

    System.out.print("Enter the string: ");

    String text = br.readLine();

    DES obj = new DES();

    String encrypted = obj.encrypt(text);

    String decrypted = obj.decrypt(encrypted);

    System.out.println("\nOriginal Text : " + text);
    System.out.println("Encrypted Text : " + encrypted);
    System.out.println("Decrypted Text : " + decrypted);
}
}

```

Output:

Enter the string: hello

Original Text : hello

Encrypted Text : 3Zkcjz0oXkg=

Decrypted Text : hello

EXPERIMENT – 5

AIM: Write a C/JAVA program to implement the Blowfish algorithm logic.

DESCRIPTION:

Blowfish is a symmetric key block cipher. Blowfish provides a good encryption rate in software and no effective cryptanalysis of it has been found to date.

Blowfish has a 64bit block size and a variable key length from 32bits up to 448bits. It is a 16-round Feistel cipher and uses large key-dependent s-boxes.

ALGORITHM:

1. Blowfish has a 64-bit block size and a variable key length from 30bits up to 48bits.
2. It is a 16-round Feistel cipher and uses large key dependent s-boxes.
3. There are 5 sub key-arrays. One 18-entry p-array and four 256-entry s-boxes. 4. Every round r consists of 4 actions.
 - a) XOR the left half of the data with the 'r'th p-array entry.
 - b) Use the XORed data as input for Blowfish algorithm. c) F-function's output with the right half (R) of the data. d) Swap L and R.
5. The F-function splits the 32bits into four 8-bits quarters and uses the quarters as input to the s-boxes.
6. The s-boxes accept 8-bit input and produce 32-bit output. The outputs are added modulo 2 power 32 and XORed to produce the final 32-bit output.

PROGRAM:

```
import java.util.Base64;
import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;

public class BlowfishDemo {

    private static final String ALGORITHM = "Blowfish";

    // Encryption
    public static String encrypt(String plainText, String key) throws Exception {

        SecretKeySpec secretKey = new SecretKeySpec(key.getBytes(), ALGORITHM);
```

```

Cipher cipher = Cipher.getInstance(ALGORITHM);
cipher.init(Cipher.ENCRYPT_MODE, secretKey);

byte[] encrypted = cipher.doFinal(plainText.getBytes());

return Base64.getEncoder().encodeToString(encrypted);
}

// Decryption
public static String decrypt(String cipherText, String key) throws Exception {

    SecretKeySpec secretKey = new SecretKeySpec(key.getBytes(), ALGORITHM);

    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.DECRYPT_MODE, secretKey);

    byte[] decrypted = cipher.doFinal(Base64.getDecoder().decode(cipherText));

    return new String(decrypted);
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    try {

        System.out.print("Enter Plain Text: ");
        String plainText = sc.nextLine();

        System.out.print("Enter Secret Key (4 to 56 characters): ");
        String key = sc.nextLine();

        String encryptedText = encrypt(plainText, key);

        System.out.println("\nEncrypted Text : " + encryptedText);

        String decryptedText = decrypt(encryptedText, key);

        System.out.println("Decrypted Text : " + decryptedText);

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }

    sc.close();
}
}

```


OUTPUT:

Enter Plain Text: Hello World

Enter Secret Key (4 to 56 characters): mysecretkey

Encrypted Text : CkVf9Q9M3rYl9gPj9Kj5WA==

Decrypted Text : Hello World

Initialization Vector of the Cipher: dI1MXzW97oQ=

Contents of inputFile.txt: Hello World

Contents of outputFile.txt: ùJÖ~ Nål“

EXPERIMENT - 6

AIM: Write a C/JAVA program to implement the Rijndael algorithm logic.

DESCRIPTION:

The more popular and widely adopted symmetric encryption algorithm likely to be encountered now-a-days in the Advanced Encryption Standard(AES). It is performed at least 6 times faster than the triple DES.

ALGORITHM:

1. Derive the set of round keys from the cipher key
2. Initialize the state array with the block data(plain text)
3. Add the initial round key to the starting state array 4.perform nine rounds of state manipulation
5. Perform the tenth and final round of state manipulation
6. Copy the final state array out as the encrypted data(cipher text)
7. Stop

PROGRAM:

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class RijndaelAES {

    // AES Key (16 bytes = 128 bits)
    private static final String key = "1234567890abcdef";

    // Encryption Method
    public static String encrypt(String plainText) throws Exception {
        SecretKeySpec secretKey = new SecretKeySpec(key.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance("AES");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);

        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());

        return Base64.getEncoder().encodeToString(encryptedBytes);
    }
}
```

```
// Decryption Method
public static String decrypt(String encryptedText) throws Exception {
    SecretKeySpec secretKey = new SecretKeySpec(key.getBytes(), "AES");

    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, secretKey);

    byte[] decodedBytes = Base64.getDecoder().decode(encryptedText);

    byte[] decryptedBytes = cipher.doFinal(decodedBytes);

    return new String(decryptedBytes);
}

public static void main(String[] args) {
    try {
        String plainText = "HelloRijndael";

        System.out.println("Original Text : " + plainText);

        String encrypted = encrypt(plainText);
        System.out.println("Encrypted Text: " + encrypted);

        String decrypted = decrypt(encrypted);
        System.out.println("Decrypted Text: " + decrypted);

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
```

OUTPUT:

```
Original Text : HelloRijndael
Encrypted Text: 4ZP6jLwNqQW4k5u4J0gP2A==
Decrypted Text: HelloRijndael
```

EXPERIMENT – 7

AIM: Write the RC4 logic in Java Using Java cryptography; encrypt the text “Hello world” using Blowfish. Create your own key using Java key tool.

DESCRIPTION:

RC4 is a stream cipher, symmetric key algorithm. The same algorithm can be used for encryption and decryption as the data stream is simply XORed with the generated key sequence. The key stream is completely independent of the plain text used. It uses a variable length key from 1 to 256 bit to initialize a 256-bit state table. The state table is used for subsequent generation of pseudo-random bits and then to generate a pseudo-random stream which is XORed with the plain text to give the cipher text.

ALGORITHM:

1. Get the data to be encrypted and the selected key
2. create two string arrays
3. Initiate one array with numbers from 0 to 255
4. Fill the other array with the selected key
5. Randomize the first array depending on the array of the key
6. Randomize the first array within itself to generate the final key stream
7. XOR the final key stream with the data to be encrypted to give the cipher text.

PROGRAM:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.swing.JOptionPane;
import java.util.Base64;

public class BlowFishCipher {

    public static void main(String[] args) throws Exception {

        // Create a key generator
        KeyGenerator keygenerator = KeyGenerator.getInstance("Blowfish");

        // Generate secret key
        SecretKey secretkey = keygenerator.generateKey();
```

```

// Create cipher
Cipher cipher = Cipher.getInstance("Blowfish");

// Read input
String inputText = JOptionPane.showInputDialog("Input your message:");

// Encrypt
cipher.init(Cipher.ENCRYPT_MODE, secretkey);
byte[] encrypted = cipher.doFinal(inputText.getBytes());

// Decrypt
cipher.init(Cipher.DECRYPT_MODE, secretkey);
byte[] decrypted = cipher.doFinal(encrypted);

// Display result
JOptionPane.showMessageDialog(
    null,
    "Encrypted Text: " + Base64.getEncoder().encodeToString(encrypted)
    + "\n\nDecrypted Text: " + new String(decrypted)
);

System.exit(0);
}
}

```

OUTPUT:

If the input is:

HELLO

Output:

Encrypted Text:

wO6Y5mB8Kk8=

Decrypted Text:

HELLO

Input your message: Hello world

Encrypted text: 3ooo&&(*&*4r4

Decrypted text: Hello world

EXPERIMENT - 8

AIM: Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript.

DESCRIPTION:

Diffie-Hellman key exchange(DH) is a method of securely exchanging cryptographic keys over a public channel and was one of the first public-key protocols as originally conceptualized by Ralph Merkle and named after Whitfield Diffie and Martin Hellman.

ALGORITHM:

1. Select prime number i.e. q
2. Find the primitive root of q i.e. x
3. Assume private key for user 'A' is x . Calculating public key for user A is $Y_A = \alpha^{x_A} \bmod q$
4. Assuming private key is X_B . Calculating public key is Y_B , $Y_B = \alpha^{X_B} \bmod q$
5. Generating secret key K , $K = (Y_A)^{X_B} \bmod q$, $K = (Y_B)^{X_A} \bmod q$
6. Stop

PROGRAM:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Diffie-Hellman Key Exchange</title>  
    <style>  
      body {  
        font-family: Arial, sans-serif;  
        margin: 30px;  
      }  
      input {  
        width: 80px;  
        margin: 5px;  
      }  
      button {  
        padding: 8px 15px;  
        margin-top: 10px;  
      }  
      #output {  
        margin-top: 20px;  
        background: #f2f2f2;  
        padding: 15px;  
        border-radius: 5px;
```

```

    }
    </style>
</head>
<body>

<h2>Diffie-Hellman Key Exchange</h2>

<label>Prime Number (p):</label>
<input type="number" id="p" value="23"><br>

<label>Primitive Root (g):</label>
<input type="number" id="g" value="5"><br>

<label>Alice's Private Key (a):</label>
<input type="number" id="a" value="6"><br>

<label>Bob's Private Key (b):</label>
<input type="number" id="b" value="15"><br>

<button onclick="diffieHellman()">Generate Shared Key</button>

<div id="output"></div>

<script>
function power(base, exponent, mod) {
    let result = 1;
    base = base % mod;

    while (exponent > 0) {
        if (exponent % 2 === 1)
            result = (result * base) % mod;

        exponent = Math.floor(exponent / 2);
        base = (base * base) % mod;
    }
    return result;
}

function diffieHellman() {

    let p = parseInt(document.getElementById("p").value);
    let g = parseInt(document.getElementById("g").value);
    let a = parseInt(document.getElementById("a").value);
    let b = parseInt(document.getElementById("b").value);

    // Public Keys
    let A = power(g, a, p);
    let B = power(g, b, p);

```

```
// Shared Secret Keys
let secretAlice = power(B, a, p);
let secretBob = power(A, b, p);

document.getElementById("output").innerHTML =
  "<h3>Results</h3>" +
  "<p>Alice's Public Key (A): <b>" + A + "</b></p>" +
  "<p>Bob's Public Key (B): <b>" + B + "</b></p>" +
  "<p>Alice's Shared Secret: <b>" + secretAlice + "</b></p>" +
  "<p>Bob's Shared Secret: <b>" + secretBob + "</b></p>" +
  "<p><b>" + (secretAlice === secretBob ?
    "Key Exchange Successful!" :
    "Key Exchange Failed!") + "</b></p>";
}
</script>

</body>
</html>
```

OUTPUT:

Sample Output

Using the default values:

Prime Number (p) = **23**

Primitive Root (g) = **5**

Alice's Private Key = **6**

Bob's Private Key = **15**

The output will be:

Alice's Public Key (A): 8

Bob's Public Key (B): 19

Alice's Shared Secret: 2

Bob's Shared Secret: 2

Key Exchange Successful!

Public key is: javax.crypto.spec.DHPublicKeySpec@5afd29

Public key is: javax.crypto.spec.DHPublicKeySpec@9971ad

EXPERIMENT - 9

AIM: Calculate the message digest of a text using the SHA-1 algorithm in JAVA.

DESCRIPTION:

In cryptography, SHA-1(Secure Hashing Algorithm) is a cryptographic hash function which takes an input and produces a 160-bit(20-byte) hash value known as message digest typically rendered as a hexadecimal number, 40 digits long.

ALGORITHM:

1. Pad the bit 00... so that length of plain text is 128<Multiple of 1024 bits
2. Append 128 bits representing of original text such that length=Multiple of 1024 bits
3. Initiate the buffers(a, b, c, d, e, f, g and h). Each buffer size=64 bits in hexadecimal
4. Process each block of plain text in 80 rounds 5.output buffers is Hash code which has length of 1 bit.

PROGRAM:

```
import java.security.MessageDigest;
import java.util.Scanner;

public class SHA1Digest {

    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            // Read input text
            System.out.print("Enter the text: ");
            String text = sc.nextLine();

            // Create SHA-1 MessageDigest instance
            MessageDigest md = MessageDigest.getInstance("SHA-1");

            // Perform the hash computation
            byte[] hashBytes = md.digest(text.getBytes("UTF-8"));

            // Convert byte array to hexadecimal format
            StringBuilder hexString = new StringBuilder();
            for (byte b : hashBytes) {
                hexString.append(String.format("%02x", b));
            }
        }
    }
}
```

```
Display the SHA-1 digest
    System.out.println("SHA-1 Message Digest:");
    System.out.println(hexString.toString());

    sc.close();

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

OUTPUT:

Input:

Enter the text: Hello World

Output:

SHA-1 Message Digest:

0a4d55a8d778e5022fab701977c5d840bbc486d0

EXPERIMENT - 10

AIM: Calculate the message digest of a text using the MD5 algorithm in JAVA.

DESCRIPTION:

The MD5 message-digest algorithm is widely a used hash function producing a 128-bit hash value. Although MD5 was initially designed to be as a cryptographic hash function, it has been found to suffer from extensive vulnerabilities. It can still be used as a checksum to verify data integrity, but only against unintentional corruption. It remains suitable for other non-cryptographic purposes, for example for determining the partition for a particular key in a partitioned database.

ALGORITHM:

1. The input message is broken up into chunks of 512-bit blocks(sixteen 32-bit words), the message is padded so that its length is divisible by 512
2. First a single bit,1 is appended to the end of message and followed by as many as zeros required to bring the length of the message upto 64 bits fewer than a multiple of 512
3. Remaining bits are filled up with 64 bits representing length of original message, modulo 264
4. MD5 operates on 128-bit state, divided into four 32-bit words, denoted A, B, C and D
5. Algorithm then uses 512-bit message block to modify the state

PROGRAM:

```
import java.security.MessageDigest;
import java.util.Scanner;

public class MD5Digest {
    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        // Read input text
        System.out.print("Enter the text: ");
        String text = sc.nextLine();

        // Create MD5 MessageDigest object
        MessageDigest md = MessageDigest.getInstance("MD5");

        // Convert text into bytes and update digest
        md.update(text.getBytes());
```

```

// Generate message digest
byte[] digest = md.digest();

// Convert byte array into hexadecimal format
StringBuilder hexString = new StringBuilder();

for (byte b : digest) {
    String hex = Integer.toHexString(0xff & b);
    if (hex.length() == 1)
        hexString.append('0');
    hexString.append(hex);
}

// Display MD5 hash
System.out.println("MD5 Message Digest: " + hexString.toString());

sc.close();
}
}

```

Sample Output

Enter the text: Hello World

MD5 Message Digest: b10a8db164e0754105b7a99be72e3fe5

